



**NOMBRE DEL ESTUDIANTE:**

Francisco Xavier Gil Ginez

**DOCENTE:**

José Miguel Carrera Pacheco

**MATERIA:**

Desarrollo Web Profesional

**CUATRIMESTRE ENERO-ABRIL**

**2026**

## **1. Comportamiento de un formulario accesible**

La accesibilidad web es el conjunto de principios y técnicas que garantizan que los sitios y aplicaciones puedan ser utilizados por personas con diversas capacidades y contextos tecnológicos. En el contexto de los formularios de login, la accesibilidad involucra aspectos visuales, auditivos, motores y cognitivos (W3C, 2018).

### **1.1 Estándares WCAG 2.1**

Las WCAG 2.1 establecen cuatro principios fundamentales para la accesibilidad web, conocidos por el acrónimo POUR: Perceptible, Operable, Comprensible y Robusto. Aplicados a un formulario de login, estos principios se traducen en los siguientes requisitos concretos (W3C, 2018):

- **Perceptible:** Todos los elementos del formulario deben poder ser percibidos por los usuarios, independientemente del canal sensorial que utilicen. Esto implica que los campos de texto deben contar con etiquetas descriptivas asociadas mediante el atributo `for/id`, que los mensajes de error sean anunciados por lectores de pantalla a través de regiones ARIA live, y que el contraste entre el texto y el fondo cumpla con una relación mínima de 4.5:1 para texto normal y 3:1 para texto grande.
- **Operable:** El formulario debe ser completamente operable con teclado. Esto incluye que el orden de tabulación sea lógico y predecible, que el foco del teclado sea siempre visible (indicador de foco claramente distinguible), y que no existan trampas de teclado que impidan al usuario navegar libremente por la interfaz.
- **Comprensible:** Las instrucciones y los mensajes de error deben ser claros y en lenguaje sencillo. Los campos deben incluir sugerencias de formato cuando sea necesario, como 'Introduce tu correo electrónico (ejemplo@dominio.com)', y los errores deben explicar tanto qué salió mal como cómo corregirlo.

- Robusto: El código HTML del formulario debe ser semánticamente correcto para garantizar compatibilidad con tecnologías de asistencia actuales y futuras, incluyendo lectores de pantalla como NVDA, JAWS o VoiceOver.

## **1.2 Etiquetas y atributos semánticos**

El uso correcto de etiquetas HTML semánticas es la base de la accesibilidad en formularios. El elemento `<label>` debe estar asociado explícitamente a cada campo mediante el atributo `for` que coincida con el `id` del `input` correspondiente. Alternativamente, es posible envolver el `input` dentro del `label` para una asociación implícita. En ningún caso debe sustituirse el `label` por el atributo `placeholder`, ya que este desaparece al comenzar a escribir y no es interpretado de manera consistente por todas las tecnologías de asistencia (MDN Web Docs, 2023).

Los atributos ARIA (Accessible Rich Internet Applications) complementan la semántica nativa de HTML. En formularios de login, los más relevantes son: `aria-label` para proporcionar nombres accesibles cuando no es posible usar un `label` visible; `aria-describedby` para asociar descripciones o mensajes de ayuda adicionales; `aria-invalid='true'` para indicar que un campo contiene un valor inválido; y `aria-live='polite'` o `aria-live='assertive'` en contenedores de mensajes de error para que sean anunciados automáticamente por lectores de pantalla cuando aparezcan (W3C, 2019).

## **1.3 Navegación y control por teclado**

La navegación exclusiva con teclado es un requisito crítico tanto para usuarios con discapacidades motoras como para usuarios avanzados que prefieren no usar el ratón. Un formulario de login accesible debe garantizar (WebAIM, 2023):

- El orden de tabulación sigue la secuencia lógica de la interfaz, generalmente de arriba hacia abajo y de izquierda a derecha, sin saltos inesperados producidos por valores de `tabindex` incorrectos.

- El indicador de foco del teclado (focus ring) es visible en todos los elementos interactivos: campos, botones y enlaces. No debe ocultarse con outline: none sin ofrecer un estilo alternativo igualmente visible.
- El formulario puede enviarse presionando Enter desde cualquiera de sus campos de texto, sin necesidad de hacer clic en el botón de envío.
- Los errores de validación muestran el foco automáticamente en el primer campo con error, o en el resumen de errores situado al inicio del formulario, para que el usuario de teclado pueda identificarlos y corregirlos sin tener que navegar toda la página.
- Los modales de error o confirmación atrapan el foco dentro de sí mismos (focus trap) mientras están abiertos y lo devuelven al elemento que los activó al cerrarse.

## **1.4 Diseño visual inclusivo**

Más allá de la funcionalidad técnica, el diseño visual influye directamente en la accesibilidad cognitiva. Los formularios de login deben emplear tipografía legible (tamaño mínimo de 14px para texto de interfaz), suficiente espaciado entre elementos, campos con dimensiones generosas (altura mínima de 44px según las guías de Apple y Google para objetivos táctiles), y un orden visual coherente con la estructura semántica subyacente. El uso de color como único medio para transmitir información, por ejemplo marcar un campo de error únicamente cambiando su borde de negro a rojo, viola el criterio 1.4.1 de las WCAG y excluye a usuarios con daltonismo (W3C, 2018).

## **2. Errores comunes en formularios de login**

A pesar de que los estándares y las buenas prácticas están bien documentados, los formularios de autenticación continúan presentando vulnerabilidades recurrentes tanto en el plano de la seguridad como en el de la accesibilidad. OWASP clasifica muchas de estas fallas dentro del Top 10 de vulnerabilidades más críticas de aplicaciones web (OWASP, 2021).

### **2.1 Errores de seguridad**

### **2.1.1 Ausencia de protección contra fuerza bruta**

Uno de los errores más frecuentes y graves es la falta de limitación en el número de intentos de inicio de sesión. Sin este control, un atacante puede realizar miles de intentos automatizados para adivinar contraseñas. Las medidas recomendadas incluyen: bloqueo progresivo de cuentas tras un número configurable de intentos fallidos (con notificación al usuario legítimo), introducción de retardos exponenciales entre intentos, implementación de CAPTCHA adaptativo y vigilancia de patrones de acceso anómalo mediante sistemas de detección de intrusiones (OWASP, 2021).

### **2.1.2 Transmisión insegura de credenciales**

La transmisión de credenciales sin cifrado TLS/SSL es un error que expone inmediatamente las contraseñas de los usuarios a ataques de tipo man-in-the-middle. Toda comunicación que involucre datos de autenticación debe realizarse exclusivamente sobre HTTPS, con certificados válidos y actualizados. Adicionalmente, el encabezado HTTP Strict-Transport-Security (HSTS) debe configurarse para forzar conexiones seguras incluso cuando el usuario introduce la URL sin el prefijo 'https://' (NIST, 2020).

### **2.1.3 Almacenamiento inseguro de contraseñas**

Almacenar contraseñas en texto plano o con algoritmos de hash débiles como MD5 o SHA-1 es una práctica gravemente insegura. El estándar actual exige el uso de funciones de hashing adaptativas diseñadas específicamente para contraseñas, como bcrypt, scrypt o Argon2id. Estas funciones incorporan un factor de trabajo (work factor) que permite ajustar la carga computacional a medida que el hardware evoluciona, manteniendo el costo de ataques por fuerza bruta elevado. También es imprescindible el uso de un salt único y aleatorio por contraseña para prevenir el uso de tablas de rainbow precomputadas (NIST, 2020).

### **2.1.4 Enumeración de usuarios**

Los mensajes de error que distinguen entre 'el usuario no existe' y 'la contraseña es incorrecta' permiten a un atacante determinar qué nombres de usuario son válidos en el sistema, reduciendo significativamente el espacio de búsqueda en ataques de fuerza

bruta. La práctica correcta es devolver siempre el mismo mensaje genérico ante cualquier fallo de autenticación, independientemente de si el error reside en el nombre de usuario o en la contraseña. Se desarrolla con mayor profundidad en la sección 4 de este documento (OWASP, 2021).

### **2.1.5 Tokens de sesión predecibles o inseguros**

La generación de tokens de sesión con valores predecibles o de baja entropía facilita ataques de adivinación o secuestro de sesión. Los tokens deben generarse con generadores de números aleatorios criptográficamente seguros (CSPRNG), tener una longitud mínima de 128 bits y renovarse completamente tras cada inicio de sesión exitoso para prevenir ataques de fijación de sesión (Stuttard & Pinto, 2011).

## **2.2 Errores de accesibilidad**

### **2.2.1 Ausencia de etiquetas en campos de formulario**

Uno de los errores de accesibilidad más comunes es la omisión de etiquetas `<label>` asociadas a los campos de texto, confiando únicamente en el atributo placeholder como guía para el usuario. Esto es problemático porque el placeholder desaparece al escribir, tiene contraste insuficiente en muchos navegadores y no es consistentemente interpretado como etiqueta por todas las tecnologías de asistencia (MDN Web Docs, 2023).

### **2.2.2 Indicadores de foco ocultos**

La tendencia al diseño minimalista ha llevado a muchos desarrolladores a eliminar el indicador de foco nativo del navegador mediante la regla CSS `outline: none` sin ofrecer un reemplazo visible. Esto hace que el formulario sea imposible de navegar para usuarios que dependen del teclado, ya que no pueden saber qué elemento está actualmente enfocado (WebAIM, 2023).

### **2.2.3 Mensajes de error no anunciados**

Cuando los mensajes de error aparecen dinámicamente en la página sin notificación a lectores de pantalla, los usuarios ciegos o con baja visión pueden no ser conscientes de que ha ocurrido un error. La solución técnica implica el uso de la API ARIA Live Regions con el atributo `aria-live='assertive'` para mensajes de error críticos que requieren atención inmediata, o `'polite'` para mensajes informativos que pueden esperar a que el usuario termine su acción actual (W3C, 2019).

#### **2.2.4 Falta de autocompletado accesible**

No configurar correctamente el atributo `autocomplete` en los campos de login perjudica a usuarios con dificultades motoras o cognitivas que se benefician de que el navegador complete automáticamente sus credenciales. Los valores estándar son `autocomplete='username'` para el campo de usuario y `autocomplete='current-password'` para el campo de contraseña (HTML Living Standard, 2023). Esto también mejora la integración con gestores de contraseñas, que son una herramienta de seguridad recomendada por NIST (2020).

### **3. Manejo seguro de sesiones**

Una vez que el usuario se ha autenticado correctamente, la seguridad de la sesión establece cuánto tiempo ese estado de autenticación se mantiene, cómo se protege frente a accesos no autorizados y cómo se termina de manera segura. Las vulnerabilidades en la gestión de sesiones figuran entre las más explotadas en aplicaciones web reales (Stuttard & Pinto, 2011).

#### **3.1 Generación y almacenamiento de tokens de sesión**

Como se mencionó anteriormente, los tokens de sesión deben ser generados usando CSPRNG y tener suficiente longitud para resistir ataques por fuerza bruta. En entornos web, los tokens suelen almacenarse en cookies del navegador. Para proteger estas cookies es imprescindible configurar los siguientes atributos (OWASP, 2021):

- HttpOnly: Impide el acceso al token desde JavaScript del lado del cliente, mitigando el riesgo de robo de sesión mediante ataques Cross-Site Scripting (XSS).
- Secure: Garantiza que la cookie solo se transmita sobre conexiones HTTPS, previniendo su exposición en canales no cifrados.
- SameSite=Strict o SameSite=Lax: Protege contra ataques Cross-Site Request Forgery (CSRF) al restringir cuándo se envía la cookie en solicitudes originadas desde otros dominios.
- Path y Domain correctamente delimitados: Reducen el alcance de la cookie al mínimo necesario, aplicando el principio de mínimo privilegio.

### **3.2 Ciclo de vida de la sesión**

Una gestión segura de sesiones implica un control riguroso sobre su ciclo de vida completo. Las prácticas recomendadas incluyen (NIST, 2020):

- Regeneración del ID de sesión tras la autenticación exitosa, para prevenir ataques de fijación de sesión (session fixation) donde el atacante predetermina el identificador que usará la víctima.
- Expiración por tiempo de inactividad: Las sesiones deben invalidarse automáticamente tras un período de inactividad configurable según el nivel de sensibilidad de la aplicación. NIST SP 800-63B recomienda reautenticación tras 30 minutos de inactividad para aplicaciones de seguridad media.
- Expiración absoluta: Independientemente de la actividad, las sesiones deben tener un tiempo máximo de vida para limitar la ventana de exposición ante un token comprometido.
- Cierre de sesión efectivo: Al hacer logout, el servidor debe invalidar el token de sesión en el lado del servidor y eliminar la cookie del cliente. Un cierre de sesión que solo borra la cookie del cliente sin invalidar el token en el servidor no proporciona protección real.



### 3.3 Tokens JWT y autenticación sin estado

Los JSON Web Tokens (JWT) se han popularizado como alternativa a las sesiones basadas en cookies para arquitecturas sin estado (stateless). Sin embargo, su uso incorrecto introduce vulnerabilidades graves. Los puntos críticos a considerar son (OWASP, 2021):

- Uso del algoritmo de firma correcto: El algoritmo 'none' no debe aceptarse nunca. Se recomienda RS256 (RSA con SHA-256) para firma asimétrica o HS256 con claves suficientemente largas para firma simétrica.
- Validación de todos los claims: Deben validarse exp (expiración), iat (emisión), nbf (no antes de) e iss (emisor) para prevenir el uso de tokens manipulados o expirados.
- Almacenamiento seguro en el cliente: Los JWT no deben almacenarse en localStorage por su vulnerabilidad a XSS. Se prefiere el uso de cookies HttpOnly, con la conciencia de que esto introduce la necesidad de protección CSRF.
- Mecanismo de revocación: A diferencia de las sesiones del servidor, los JWT no pueden invalidarse individualmente a menos que se mantenga una lista de revocación. Este es uno de sus principales inconvenientes en escenarios donde el logout inmediato es crítico.

### 3.4 Autenticación multifactor (MFA)

La autenticación multifactor añade una capa adicional de seguridad al requerir que el usuario demuestre su identidad mediante dos o más factores: algo que sabe (contraseña), algo que tiene (token TOTP, SMS) o algo que es (biometría). NIST SP 800-63B clasifica los autenticadores por niveles de seguridad (AAL - Authenticator Assurance Level) y recomienda MFA para cualquier aplicación que maneje datos personales o financieros. Las aplicaciones TOTP (como Google Authenticator o Authy) proporcionan un equilibrio adecuado entre seguridad y usabilidad, mientras que las llaves de seguridad físicas (FIDO2/WebAuthn) ofrecen el nivel más alto de protección contra phishing (NIST, 2020).

## **4. Errores de autenticación sin filtrar información sensible**

La forma en que una aplicación comunica los errores de autenticación tiene implicaciones directas tanto en la seguridad como en la experiencia del usuario. El principio general es que los mensajes de error deben ser informativos para el usuario legítimo sin revelar información que pueda ser aprovechada por un atacante (OWASP, 2021).

### **4.1 El problema de la enumeración de usuarios**

Cuando un sistema muestra mensajes de error diferenciados según si el fallo se debe a un nombre de usuario inexistente o a una contraseña incorrecta, proporciona a los atacantes la capacidad de enumerar usuarios válidos. Esta información es valiosa para ataques dirigidos de fuerza bruta o de ingeniería social. El mismo problema puede ocurrir de manera más sutil a través del tiempo de respuesta: si el sistema verifica primero si el usuario existe y solo entonces comprueba la contraseña, el tiempo de respuesta puede ser diferente según si el usuario es válido o no, permitiendo ataques de temporización (timing attacks) (Stuttard & Pinto, 2011).

La solución es implementar una lógica de autenticación que tenga un tiempo de respuesta constante independientemente del tipo de error, utilizando comparaciones de contraseña con tiempo constante y retornando siempre el mismo mensaje genérico.

### **4.2 Mensajes recomendados y prohibidos**

La siguiente tabla conceptual resume los mensajes que deben evitarse y sus alternativas seguras:

- PROHIBIDO: 'El usuario estudiante@ejemplo.com no existe en nuestro sistema' — Confirma la inexistencia del usuario.
- PROHIBIDO: 'La contraseña es incorrecta. ¿Olvidaste tu contraseña?' — Confirma que el usuario sí existe.
- PROHIBIDO: 'Tu cuenta ha sido bloqueada por 10 intentos fallidos' — Revela la política exacta de bloqueo.

- RECOMENDADO: 'Las credenciales introducidas no son correctas. Por favor, verifica tu nombre de usuario y contraseña.' — Neutro, aplicable a cualquier tipo de error.
- RECOMENDADO: 'Por razones de seguridad, tu cuenta ha sido temporalmente suspendida. Recibirás un correo con instrucciones.' — Informa sin revelar detalles del sistema de bloqueo.

### **4.3 Registro de errores para el servidor vs. mensajes para el cliente**

Es fundamental distinguir entre la información que se registra en los logs del servidor y la que se muestra al usuario. Los logs del servidor deben ser detallados e incluir: dirección IP del intento, timestamp, nombre de usuario utilizado (si no es considerado dato sensible por la política de privacidad), tipo de error específico y user agent del navegador. Esta información es esencial para la detección de ataques y la respuesta a incidentes (NIST, 2020).

Sin embargo, esta misma información nunca debe llegar al cliente, ya sea en el cuerpo de la respuesta HTTP, en los encabezados o en mensajes de error de la aplicación. Especialmente peligroso es exponer stack traces, mensajes de error de base de datos o detalles sobre la arquitectura interna del sistema, que pueden facilitar la explotación de vulnerabilidades adicionales.

### **4.4 Recuperación de contraseña: el eslabón frecuentemente olvidado**

El flujo de recuperación de contraseña es una extensión del proceso de autenticación y debe aplicar los mismos principios de opacidad informativa. Los errores comunes incluyen (OWASP, 2021):

- Confirmar si un email está registrado en el sistema: El mensaje siempre debe indicar que 'si la dirección existe en nuestro sistema, recibirás un correo de recuperación', independientemente de si el email es válido o no.

- Tokens de recuperación predecibles o de vida larga: Los tokens de restablecimiento de contraseña deben ser criptográficamente aleatorios, de uso único y tener una expiración corta (15-60 minutos).
- Envío de contraseñas por correo: Nunca debe enviarse la contraseña actual por correo (lo que implicaría que está almacenada en texto plano). Solo deben enviarse enlaces de restablecimiento de un solo uso.

## Referencias

*Las siguientes referencias están presentadas en formato APA 7.<sup>a</sup> edición, ordenadas alfabéticamente por apellido del autor o nombre de la organización.*

MDN Web Docs. (2023). The HTML <label> element. Mozilla. <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/label>

National Institute of Standards and Technology [NIST]. (2020). Digital identity guidelines: Authentication and lifecycle management (NIST Special Publication 800-63B). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-63b>

Open Web Application Security Project [OWASP]. (2021). OWASP Top 10:2021. OWASP Foundation. <https://owasp.org/Top10/>

Open Web Application Security Project [OWASP]. (2021). Testing for account enumeration and guessable user account (OTG-IDENT-004). OWASP Testing Guide v4.2. <https://owasp.org/www-project-web-security-testing-guide/>

Open Web Application Security Project [OWASP]. (2021). Session management cheat sheet. OWASP Cheat Sheet Series. [https://cheatsheetseries.owasp.org/cheatsheets/Session\\_Management\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html)

Stuttard, D., & Pinto, M. (2011). The web application hacker's handbook: Finding and exploiting security flaws (2nd ed.). Wiley.

WHATWG. (2023). HTML living standard: The autocomplete attribute. Web Hypertext Application Technology Working Group. <https://html.spec.whatwg.org/multipage/form-control-infrastructure.html#autofilling-form-controls:-the-autocomplete-attribute>

W3C. (2018). Web content accessibility guidelines (WCAG) 2.1. World Wide Web Consortium. <https://www.w3.org/TR/WCAG21/>